ANÁHUAC MAYAB

Nombre de la Universidad: Anáhuac Mayab

Campus: Merida

Nombre del Actividad de Aprendizaje: Coursera - Actividad 3

Nombre del alumno: Emiliano Parra - Eduardo Téllez – Santiago Loredo - Leonardo Argaez Ortega

Nombre del Docente: Lizbeth Hernández Olan

Fecha: 19/10/2026

Ruta mágica - Documentación final de requerimientos

Estructura de Datos y Algoritmos · Proyecto de grafos · Anáhuac Mayab · Versión 2.0

1. Objetivo

Desarrollar un programa de consola que obtenga la ruta más corta entre dos Pueblos Mágicos de un estado de México, modelando las carreteras como un grafo ponderado no dirigido e implementando el algoritmo de Dijkstra.

Objetivos específicos desde la materia de Estructura de Datos y Algoritmos:

- Modelar los Pueblos Mágicos de un estado como un grafo (TAD Grafo).
- Elegir y justificar un algoritmo de caminos mínimos con base en su complejidad.
- Seleccionar la estructura de datos adecuada para representar el grafo.
- Verificar el funcionamiento con casos de prueba.

2. Descripción del problema

México cuenta con 177 Pueblos Mágicos distribuidos en los 31 estados; cada estado tiene entre 2 y 12, conectados por carreteras con distancias conocidas. Este proyecto modela **un solo estado** (Puebla, Sierra Norte, 7 pueblos). El usuario introduce un pueblo de origen y uno de destino, y el sistema calcula y muestra la ruta óptima.

Elemento del grafo	Representación	Ejemplo
Vértice (nodo)	Cada Pueblo Mágico del estado	Zacatlán de las Manzanas
Arista	Carretera que conecta directamente dos pueblos	Zacatlán - Chignahuapan
Peso	Distancia en km por carretera	22 km

2.1 Supuestos y restricciones

- **Único medio de transporte: vehículo** (indicación del profesor). Las distancias son por carretera en automóvil, tomadas de Google Maps (modo auto) o de fuentes oficiales.
- El grafo es **no dirigido**: si hay carretera de A a B, también de B a A con la misma distancia.
- Todos los pesos son positivos (km); no hay pesos negativos.
- Entre dos pueblos solo hay una arista directa; si hubiera varias carreteras, se usa la más corta.
- El grafo de Puebla (Sierra Norte) es conexo: existe al menos una ruta entre cualquier par de pueblos.

3. Algoritmo elegido y justificación

Se elige **Dijkstra**. La tabla compara los tres algoritmos que propone la actividad.

Algoritmo	Tipo	Pesos negativos	Complejidad en tiempo	Se ajusta al problema
Dijkstra	Origen único	No soporta	$O((V + E) \log V)$ con cola de prioridad	Sí
Floyd-Warshall	Todos los pares	Soporta (sin ciclos negativos)	$O(V^3)$, espacio $O(V^2)$	No: calcula más de lo necesario
Bellman-Ford	Origen único	Soporta	$O(V \cdot E)$	No: más lento y sin pesos negativos que justifiquen su uso

Justificación:

- Los pesos son distancias en km, siempre positivos, por lo que se cumple la condición de Dijkstra.
- La consulta es de un origen a un destino, no de todos los pares.
- El grafo es disperso (7 vértices y 10 aristas frente a 21 posibles), donde la cola de prioridad es eficiente.
- Es el algoritmo que usa el ejemplo de ejecución esperada de la actividad.

4. Representación del grafo

Se usa **lista de adyacencia** con `HashMap<String, List<Edge>>`, porque el grafo es disperso. La matriz de adyacencia de referencia del proyecto se usa solo para verificar los datos.

Criterio	Lista de adyacencia	Matriz de adyacencia
Espacio	$O(V + E)$	$O(V^2)$
Recorrer los vecinos de un vértice	O(grado del vértice)	O(V)
Saber si existe una arista A-B	O(grado del vértice)	O(1)
Mejor para	Grafos dispersos	Grafos densos

Cada `Edge` guarda el pueblo vecino y la distancia en km. Como el grafo es no dirigido, cada línea del archivo agrega dos entradas: A hacia B y B hacia A.

5. Datos del estado: Puebla (Sierra Norte)

#	Pueblo Mágico	Clave
1	Zacatlán de las Manzanas	ZAC
2	Chignahuapan	CHI
3	Tetela de Ocampo	TET

#	Pueblo Mágico	Clave
4	Cuetzalan del Progreso	CUE
5	Tlatlauquitepec	TLA
6	Xicotepec de Juárez	XIC
7	Pahuatlán	PAH

5.1 Matriz de adyacencia de referencia (km)

El símbolo ∞ indica que no existe conexión directa entre los dos pueblos.

	ZAC	CHI	TET	CUE	TLA	XIC	PAH
ZAC	0	22	48	∞	∞	55	60
CHI	22	0	42	∞	90	∞	∞
TET	48	42	0	68	∞	∞	∞
CUE	∞	∞	68	0	38	115	∞
TLA	∞	90	∞	38	0	∞	∞
XIC	55	∞	∞	115	∞	0	32
PAH	60	∞	∞	∞	∞	32	0

5.2 Archivo de entrada

Una arista por línea con el formato `Pueblo1 Pueblo2 distancia_km`. Los nombres no llevan espacios (se usa guion bajo) y las líneas que inician con `#` son comentarios.

```
# Pueblos Magicos de Puebla -- Distancias en km
Zacatlan Chignahuapan 22
Zacatlan Tetela_de_Ocampo 48
Chignahuapan Tetela_de_Ocampo 42
Tetela_de_Ocampo Cuetzalan 68
Cuetzalan Tlatlauquitepec 38
Zacatlan Xicotepec 55
Xicotepec Pahuatlan 32
Pahuatlan Zacatlan 60
Chignahuapan Tlatlauquitepec 90
Xicotepec Cuetzalan 115
```

6. Estructuras de datos y análisis del algoritmo

6.1 Estructuras de datos utilizadas

Estructura	Uso en el programa	Costo clave
HashMap<String, List<Edge>>	Almacena el grafo (lista de adyacencia)	Acceso a un vértice O(1) promedio
PriorityQueue (montículo mínimo)	Elige siempre el vértice con menor distancia acumulada	Insertar y extraer O(log n)
HashMap<String, Double> dist	Distancia mínima conocida desde el origen a cada vértice	O(1) promedio
HashMap<String, String> previo	Vértice anterior en el camino más corto	O(1) promedio
Pila o lista enlazada	Reconstruye la ruta del destino al origen y la invierte	O(V)

6.2 Pseudocódigo de Dijkstra

```
DIJKSTRA(G, origen, destino)
  para cada vértice v en G:
    dist[v] = ∞
    previo[v] = nulo
  dist[origen] = 0
  cola.insertar(origen, 0)
  mientras cola no esté vacía:
    u = cola.extraerMin()
    si u ya fue visitado: continuar
    marcar u como visitado
    si u == destino: terminar
    para cada arista (u, v, w) en G.adyacentes(u):
      si dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        previo[v] = u
        cola.insertar(v, dist[v])
  si dist[destino] == ∞: no existe ruta
  reconstruir la ruta siguiendo previo desde destino hasta origen
```

6.3 Complejidad

- Tiempo: O((V + E) log V), por las inserciones y extracciones en la cola de prioridad.
- Espacio: O(V + E) para la lista de adyacencia más O(V) para `dist`, `previo` y la cola.
- En este proyecto V = 7 y E = 10 (20 entradas en la lista, porque cada arista se guarda en ambos sentidos).

6.4 Traza de ejemplo: Xicotepec a Tlatlauquitepec

Paso	Vértice extraído (distancia)	Distancias actualizadas
1	XIC (0)	ZAC = 55, PAH = 32, CUE = 115
2	PAH (32)	Sin mejora (ZAC por PAH sería 92)
3	ZAC (55)	CHI = 77, TET = 103
4	CHI (77)	TLA = 167
5	TET (103)	Sin mejora (CUE por TET sería 171)
6	CUE (115)	TLA = 153 (mejora a 167)
7	TLA (153)	Destino alcanzado

Ruta resultante: XIC → CUE → TLA, 153 km.

6.5 Diseño de clases

Clase	Responsabilidad
`Edge`	Guarda el pueblo vecino y la distancia en km
`Graph`	Construye el grafo desde el archivo y devuelve los vecinos de cada pueblo
`Dijkstra`	Calcula la distancia mínima y reconstruye la ruta
`Main` (menú)	Muestra los pueblos, lee origen y destino, imprime el resultado y repite la consulta

7. Flujo de la aplicación

El programa sigue las cuatro etapas del esquema de la actividad:

1. **Datos de entrada:** se lee el archivo y se construye el grafo ponderado no dirigido.
2. **Selección:** el usuario elige el pueblo de origen y el de destino por número; el sistema indica el algoritmo (Dijkstra).
3. **Procesamiento:** se ejecuta Dijkstra con la cola de prioridad y se llenan `dist` y `previo`.
4. **Resultado:** se muestran los tramos con su distancia, la ruta completa y la distancia total (ejemplo: 153 km en 2 saltos).

Flujos alternos: entrada inválida, origen igual a destino, ausencia de ruta y error de archivo.

8. Ejemplo de ejecución esperada

La salida del programa debe seguir este formato (tomado de la actividad):

```
============================================================
      RUTA MAGICA -- PUEBLA
   Camino mas corto entre Pueblos Magicos
```

```
======================================================================

Pueblos Magicos disponibles:
  1. Zacatlan de las Manzanas
  2. Chignahuapan
  3. Tetela de Ocampo
  4. Cuetzalan del Progreso
  5. Tlatlauquitepec
  6. Xicotepec de Juarez
  7. Pahuatlan

Seleccione pueblo de ORIGEN (1-7): 6
  -> Origen: Xicotepec de Juarez

Seleccione pueblo de DESTINO (1-7): 5
  -> Destino: Tlatlauquitepec

Algoritmo utilizado: Dijkstra
Calculando ruta mas corta...

======================================================================
  [OK] RUTA ENCONTRADA
======================================================================

  Xicotepec --(115 km)--> Cuetzalan
  Cuetzalan --( 38 km)--> Tlatlauquitepec

  Ruta completa:
  Xicotepec -> Cuetzalan -> Tlatlauquitepec

  Distancia total: 153 km
======================================================================

Desea consultar otra ruta? (S/N): S
```

9. Requerimientos funcionales

ID	Requerimiento	Criterio de aceptación
RF-01	Cargar el grafo desde un archivo de texto con el formato definido.	El grafo se construye con todos los pueblos y aristas del archivo.
RF-02	Construir un grafo ponderado no dirigido con lista de adyacencia.	Cada arista existe en ambos sentidos con igual peso.
RF-03	Mostrar la lista numerada de Pueblos Mágicos disponibles.	Se muestran los 7 pueblos con número del 1 al 7.

ID	Requerimiento	Criterio de aceptación
RF-04	Solicitar el pueblo de origen por número.	El sistema confirma el nombre del origen elegido.
RF-05	Solicitar el pueblo de destino por número.	El sistema confirma el nombre del destino elegido.
RF-06	Validar las entradas del usuario.	Un número fuera de rango o un texto no numérico muestra un mensaje y se vuelve a pedir.
RF-07	Calcular la ruta más corta con Dijkstra.	La distancia obtenida es la mínima posible del grafo.
RF-08	Mostrar cada tramo con su distancia, la ruta completa y la distancia total.	La salida sigue el formato del ejemplo de ejecución esperada.
RF-09	Manejar origen igual a destino.	Se informa distancia total de 0 km.
RF-10	Manejar la ausencia de ruta.	Se muestra un mensaje claro de que no existe ruta entre ambos pueblos.
RF-11	Mostrar el algoritmo utilizado.	Aparece "Algoritmo utilizado: Dijkstra".
RF-12	Permitir consultas repetidas.	Con S se reinicia la selección; con N el programa termina.
RF-13	Manejar errores de archivo.	Si el archivo no existe o una línea es inválida (distancia no numérica o negativa), se informa el error sin cerrar el programa de forma abrupta.

10. Requerimientos no funcionales

ID	Requerimiento
RNF-01	Usabilidad: interfaz de consola con mensajes claros y opciones numeradas.
RNF-02	Mantenibilidad: código modular (clases Edge, Graph, Dijkstra y menú) y comentado.
RNF-03	Extensibilidad: cambiar de estado solo requiere cambiar el archivo de datos.
RNF-04	Rendimiento: cada consulta responde de inmediato para hasta 12 pueblos por estado.
RNF-05	Fidelidad de datos: distancias por carretera en vehículo, de Google Maps u otras fuentes oficiales.

11. Casos de prueba

ID	Origen	Destino	Resultado esperado
CP-01	Xicotepec	Tlatlauquitepec	153 km: XIC → CUE → TLA
CP-02	Pahuatlán	Tlatlauquitepec	172 km: PAH → ZAC → CHI → TLA
CP-03	Zacatlán	Chignahuapan	22 km (arista directa)
CP-04	Zacatlán	Zacatlán	0 km
CP-05	Tlatlauquitepec	Pahuatlán	172 km (simetría del grafo no dirigido)
CP-06	Entrada "9" o "abc"	-	Mensaje de error y nueva solicitud
CP-07	Pueblo sin conexión (grafo modificado)	Cualquiera	Mensaje de que no existe ruta
CP-08	Archivo inexistente	-	Mensaje de error controlado

12. Trazabilidad con la actividad

Requisito de la actividad	Dónde se cubre
Modelar un estado como grafo ponderado (vértices, aristas, pesos)	Secciones 2 y 5, RF-01, RF-02
El usuario introduce origen y destino	RF-03 a RF-06
Mostrar la ruta óptima con un algoritmo clásico	Secciones 6 y 8, RF-07, RF-08, RF-11
Elegir un algoritmo y justificarlo	Sección 3
Representar con lista o matriz de adyacencia	Sección 4
Leer el grafo desde archivo de texto	Sección 5.2, RF-01, RF-13
Grafo no dirigido salvo justificación	Sección 2.1, RF-02
Distancias reales por carretera (Google Maps u oficiales)	Sección 2.1, RNF-05
Salida igual al ejemplo de ejecución esperada	Sección 8, RF-08

13. Validación con asistente de IA

Evidencia de la actividad: el documento propio (versión 0.1), los prompts, las observaciones del asistente y esta versión final.

13.1 Prompts utilizados

Prompt 1 (validación):
Actúa como revisor de ingeniería de software. Te comparto la descripción del proyecto "Ruta mágica" (grafos, Pueblos Mágicos, materia Estructura de Datos y Algoritmos) y la documentación de requerimientos que redactamos. Revisa: (1) que cubra todo lo pedido en la descripción, (2) ambigüedades o contradicciones, (3) requerimientos faltantes o mal redactados, (4) casos de prueba faltantes, (5) si la justificación del algoritmo y la estructura de datos es correcta. Responde con una lista numerada de observaciones y una corrección propuesta para cada una.
Restricción: el único transporte es vehículo.

Prompt 2 (seguimiento):
Con base en tus observaciones, ¿qué requerimientos son críticos y cuáles opcionales? Indica cuáles de tus sugerencias se salen del alcance.

13.2 Observaciones del asistente y decisiones del equipo

Completar con las observaciones reales del asistente.

Observación del asistente	Decisión	Cambio realizado
Faltaban validaciones de entrada	Aceptada	Se añadió RF-06
No se contemplaban casos límite	Aceptada	Se añadieron RF-09 y RF-10
Faltaban criterios de aceptación y casos de prueba	Aceptada	Columna de criterios y sección 11
La justificación del algoritmo era incompleta	Aceptada	Sección 3 con complejidad de los tres algoritmos
Faltaba el análisis propio de la materia (estructuras, pseudocódigo, complejidad)	Aceptada	Sección 6
Faltaba seguir la estructura del documento de la actividad	Aceptada	Secciones 1 a 8 y 12

14. Notas

- Las distancias del ejemplo vienen del documento de la actividad; antes de entregar, verificarlas en Google Maps en modo automóvil.
- Si se cambia de estado, basta con otro archivo de datos con el mismo formato.